

Week 11 Software Lesson: Other Regression in R

Goals

The goal of this software lesson is to learn how to

- fit a log-binomial regression model and obtain estimates for the effect of a predictor in terms of log probabilities, probabilities, and risk ratios, and
- fit an ordinal regression model and obtain estimates for the effect of a predictor in terms of cumulative log odds and common odds ratios.

Required Packages

You will need to install the following packages in R for this software lesson:

- jtools
- VGAM

DATASET

In this software lesson, you will use the Obstetrics and Periodontal Therapy (OPT) dataset. The dataset, data dictionary, and dataset introduction can be found on Canvas, titled *OTP.csv*, *OPT Data Dictionary.pdf*, and *OPT Dataset Introduction.pdf*, respectively.

```
opt <- read.csv(file = file.choose(), header = TRUE)
```

LOG-BINOMIAL & ORDINAL REGRESSION

Thus far in the course, you have learned four kinds of regression models:

- *linear* regression to model the mean of a continuous outcome,
- *logistic* regression to model the log odds of a binary outcome,
- *proportional hazards* regression to model the log hazard of a time-to-event outcome, and
- *Poisson* regression to model the log count/rate of a discrete/count outcome.

This software lesson builds on what you have learned thus far, but focuses on *log-binomial* regression, which is used when you have a binary outcome, and *ordinal* regression, which is used when you have a ordinal categorical outcome.

LOG-BINOMIAL REGRESSION

Log-binomial regression is similar to logistic regression in the sense of using it for a binary outcome, but instead of modeling the log odds of the outcome, you model the log probability/risk of the outcome.

In the OPT study, suppose the researchers were interested in examining the effect of treatment of maternal periodontal disease (**Group**) on risk of preterm birth (**preterm**), after adjusting for number of missed prenatal appointments (**NumMiss**) and previous preterm births status (**Live.PTB**). In this example, **Group** is the predictor of interest, and **NumMiss** and **Live.PTB** are considered to potential confounders.

Data cleaning

EDIT 1: Creating the preterm variable

The outcome, `preterm`, needs to be created from the variable `GA.at.outcome`, which is stored as gestational age at end of pregnancy (in number of days). A preterm birth is defined as a birth with a gestational age that is less than 37 weeks (i.e., 259 days). We need to create a new indicator variable (0/1) for preterm birth.

This can be done with the `ifelse()` function. The first argument to the function is the *if* condition, followed by the *then* result, and finally the *else* result. The following code can be read as:

IF the gestational age is less than 259 days, THEN set the preterm indicator to 1, ELSE set the preterm indicator to 0.

```
opt$preterm <- ifelse(opt$GA.at.outcome < 259, 1, 0)
```

Additional Resources

For additional R resources on the `ifelse()` function, see:

- [If statements and operators \(from datacamp\)](#)

EDIT 2: Creating the NumMiss variable

The variable, `NumMiss`, needs to be created by taking the number of prenatal visits the participant was eligible for (which R read as `X..Vis.Elig`) and subtracting the number of prenatal visits the participants attended (which R read as `X..Vis.Att`)

```
opt$NumMiss <- opt$X..Vis.Elig - opt$X..Vis.Att
```

Be sure to check these variables to make sure the coding worked correctly.

Log-Binomial Regression Model

We will use the `glm()` function with the `family=binomial(link="log")` argument to specify doing log-binomial regression as opposed to some other type of generalized linear model.

```
model1 <- glm(preterm ~ Group + NumMiss + Live.PTB, data=opt, family=binomial(link="log"))  
summ(model1, confint = TRUE, ci.width = 0.95, digits = 4)
```

The model output is the same as in multiple linear regression, logistic regression, and Poisson regression, but estimated regression coefficients are in terms of log of the risk.

Estimate effect of predictors

Recall that in a log-binomial regression model, the regression coefficients represent changes in the log of the risks (i.e., log of the probabilities)

Relative Risk

- To obtain relative risk estimates, use the code you have seen thus far, but in the `summ()` function, also include the `exp = TRUE` argument to report exponentiated coefficients (i.e., relative risks) and confidence intervals:

```
summ(model1, confint = TRUE, ci.width = 0.95, digits = 4, exp = TRUE)
```

Predicted Risk

We can estimate a risk value for a particular observation/individual manually by using the equation from the model to calculate the predicted log of the risk and then exponentiating that value.

```
#prediction, by hand, using the regression coefficients  
#for an individual in the control group, 2 missed visits, and no previous preterm births:  
exp(-2.17427 + 0*-0.19425 + 2*0.11822 + 0*0.90350)
```

This can also be done automatically with the model and the `predict()` function.

```
#prediction, automated, using the predict() function  
#for an individual in the control group, with 2 appointments missed,  
#and no previous preterm births:  
pred.data <- data.frame(Group = "C", NumMiss = 2, Live.PTB = "No ")  
# Note that the space after the No in Live.PTB = "No " is essential.  
  
pred.logrisk <- predict(model1, pred.data, interval="confidence")  
  
pred.risk <- exp(pred.logrisk)  
  
pred.risk
```

Suppose we wanted to make predictions for multiple individuals. The `predict()` function allows us to this in one fell swoop. Simply create one row per individual you want to calculate a predicted risk for in a data frame (`newdata`).

```
#Make a prediction for multiple individuals  
#Individual 1: control group, 0 appointments missed, with previous preterm births  
#Individual 2: control group, 0 appointments missed, with no previous preterm births  
#Individual 3: control group, 2 appointments missed, with previous preterm births  
#Individual 4: control group, 2 appointments missed, with no previous preterm births  
#Individual 5: treatment group, 0 appointments missed, with previous preterm births  
#Individual 6: treatment group, 0 appointments missed, with no previous preterm births  
#Individual 7: treatment group, 2 appointments missed, with previous preterm births  
#Individual 8: treatment group, 2 appointments missed, with no previous preterm births  
  
newdata <- data.frame(  
  Group = c("C", "C", "C", "C", "T", "T", "T", "T"),  
  NumMiss = c(0, 0, 2, 2, 0, 0, 2, 2),  
  Live.PTB = c("Yes", "No ", "Yes", "No ", "Yes", "No ", "Yes", "No ")  
)  
  
newdata$pred.logrisk <- predict(model1, newdata)  
  
newdata$pred.risk <- exp(newdata$pred.logrisk)  
  
#store predicted risk in the data frame  
newdata
```

Unfortunately, there is no easy way to obtain confidence intervals for the prediction in R.

ORDINAL REGRESSION

Ordinal regression is used when the outcome is an ordinal categorical variable.

In the OPT study, suppose the researchers were interested in examining the effect of treatment of maternal periodontal disease (`Group`) on babies' APGAR scores (`apgarlevel`), after adjusting for number of missed prenatal appointments (`NumMiss`) and previous preterm births status (`Live.PTB`). In this example, `Group` is the predictor of interest, and `NumMiss` and `Live.PTB` are considered to potential confounders.

Data cleaning

EDIT 3: Creating the `apgarlevel` variable

The outcome, `apgarlevel`, needs to be created from the variable `Apgar1`, which is stored as a numeric variable, but it is an ordinal variable, and thus must be converted into a factor. Furthermore, it is currently scored between 0 and 10, but, scores are interpreted in the following manner:

- 7 to 10 = normal (these scores indicate no further interventions need to take place)
- 4 to 6 = low (these scores indicate low oxygen levels and the newborn should be closely monitored and encouraged to breathe)
- 0 to 3 = critically low (these scores indicate critically low oxygen levels and medical professionals should increase oxygen immediately)

To create a new factor variable with three levels, use the `cut()` function. The `cut()` function takes a numeric variable and breaks it up into categories. The arguments to the function typically include the variable that needs to be categorized, the points at which we want to split the variable into categories, and labels for each of the new categories.

```
##Create variable for Apgar1 as factor
opt$apgarlevel <- cut(opt$Apgar1,

  breaks = c(0, 3, 6, 10),
  #splits the number line into "3 or lower", "6 or lower", "10 or lower"

  include.lowest = TRUE, #includes the first break, 0, with the lowest category

  labels = c("critically low", "low", "normal"),
  ordered_result = TRUE #returns an ordered factor
)
```

Be sure to check this variable to make sure the coding worked correctly.

Additional Resources

For additional R resources on the `cut()` function, see:

- [Cut \(from R Bloggers\)](#)

Ordinal Regression Model

To fit an ordinal regression model, use the `vglm()` function from the `{VGAM}` package. Specify the model in the same way you would in `glm()`, where you specify using logistic in the `family=` argument (i.e., `family = cumulative(link = "logitlink")`). However, unlike for GLM, you also need to specify that the model is created under the proportional odds assumption (i.e., `parallel = TRUE`).

```
model2 <- vglm(apgarlevel ~ Group + NumMiss + Live.PTB,
               data = opt, family = cumulative(link = "logitlink", parallel = TRUE))

summary(model2)
```

The output is similar to what you have seen, but there is one major difference: there is multiple intercept terms. Recall that ordinal regression fits a series of equations, each with different intercepts, for different levels of the outcome variable. These intercepts are at the top of the rows in the coefficients table, labeled as `(Intercept):1`, `(Intercept):2`, etc. The intercepts are in terms of the log odds.

The main information for explanatory variables in the model are in the table under the header **Coefficients** (in terms of common log odds). The regression coefficient values are under the column **Estimate**. For categorical variables, the row will be labeled with the variable name and the level that R denoted as 1. Remember that you can also specify a reference level by reordering factors with the `relevel()` function.

The standard errors for each of the regression coefficients are under the column **Std. Error**. This value is used to create confidence intervals and *z*-test statistics for the coefficients.

In the third column, **z value** represents the *z*-test statistics for each the regression coefficient. It is found by taking the **Estimate** divided by **Std. Error**.

The last column presents the *p*-values for testing each the regression coefficient.

Estimate effect of predictors

Recall that in an ordinal regression model, the regression coefficients represent changes in the common log odds.

Common Odds Ratio

The common odds ratio between groups of an explanatory variable, which is common across all levels of the ordinal response variable, can be calculated by taking the anti-log of the regression coefficients. This information is automatically presented to you at the bottom of the output for the `vglm` model under the header **Exponentiated coefficients**. It can also be calculated by exponentiating the beta coefficients, using the `exp()` function.

You can only do this if you first extract the regression coefficients from the model output (by using the `coef()` function on the ordinal regression model object) and then exponentiate on those coefficients.

```
exp(coef(model2))
```

Similarly, the confidence interval limits can also be exponentiated and converted into confidence intervals for the common odds ratios.

```
exp(confint(model2))
```

Additional Resources

For additional R resources on ordinal regression, see:

- [Unified Approach to Ordinal \(cumulative\) and Polytomous \(multinomial\) Logistic Regressions using VGAM::vglm \(from RPubS by RStudio\)](#)